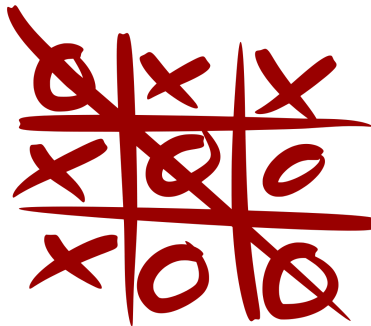


Fligoo tech interview: Tic-Tac-Toe API

Description

Tic-Tac-Toe is a game in which two players alternately put Xs and Os in compartments of a figure formed by two vertical lines crossing two horizontal lines and each tries to get a row of three Xs or three Os before the opponent does.



Assignment

The goal of this exercise is to develop an API that lets you play the tic-tac-toe game, validate the board and return the winner, a message saying it was a tie, or an error if there was some kind of problem with the input data.

The API should contain the following endpoints in order to play a game:

Create a game:

Provide an endpoint that will receive the information of two players, such as their names. The API will return an object with the game ID, player information, and which player has to play next. For example:

Input data:

```
{
  "players": [
    {"name": "me"},
    {"name": "other"}
  ],
  "starting_player": "me"
}
```

Return data:

```
{
```

```

    "game_id": 1234,
    "players": [{ "name": "me", "symbol": "X"}, { "name": "other", "symbol": "O"}]
    "movements_played": 0,
    "next_turn": "other",
    "board": [[null, null, null], [null, null, null], [null, null, null]],
    "winner": null
  }

```

Optional improvement ideas:

- How would you let the players choose their symbols instead of using X and O?
- Let the players choose who goes first instead of always choosing the first player in the list.
- How can the API validate and handle malformed inputs or invalid data?

Submit play

When a game is created, the player who goes next should be able to submit a move. In order to do that, another endpoint expects the game id, player name, row, and column. The returned data should be the board state after the current play. If the last move causes the game to end, the name of the winner should be on the "winner" field.

For example:

Input data:

```

{
  "game_id": 1234,
  "player": "me",
  "row": 1,
  "column": 2
}

```

Returned data:

```

{
  "game_id": 1234,
  "players": [{ "name": "me", "symbol": "X"}, { "name": "other", "symbol": "O"}],
  "movements_played": 2,
  "next_turn": "me",
  "board": [[null, null, null], [null, "X", "O"], [null, null, null]],
  "winner": null
}

```

Optional improvement ideas:

- How can the API validate and handle malformed inputs or invalid data? Consider the case where the same player tries to play twice in a row, or a player attempts to make a move to a finished game.

List all games

Provide an endpoint to list all the games. Finished games should also specify the winner.

Optional improvement ideas:

- How would you extend the endpoint to return only finished or unfinished games?
- What would happen if you had millions of stored games? Is there something you can do to avoid returning all of them at once?

Retrieve a single game

The API should let the user retrieve a single game by its ID.

Optional improvement ideas:

- How would you handle the case where the ID is invalid?
- Imagine the data needed when **listing** games is different from the data needed when **retrieving** a game. How would you implement that?

Delete a game

Provide a way to delete a game by ID.

Optional improvement ideas:

- How would you handle the case where the ID is invalid?
- And what about trying to delete a game that's already deleted?

Mandatory

- Technologies: The work involved should be done by using at least the following stack:
 - Python 3
 - Any API framework
 - Any database (SQLite, PostgreSQL, MySQL)
- Code versioning with Git (you are free to publish it on your own Github/Bitbucket account).
- Documentation to use the project (README, usage instructions)

Bonus

- Unit testing
- Any additional code documentation.
- Helper scripts to set up the project or install the required dependencies
- Dockerfile / docker-compose to set up the environment.

Tips

- Database modeling and endpoint definitions are up to you.
- You can use as many libraries as you want. Just make sure you can justify what are you using them for, and why.

This assessment is designed to be finished in 2 days. Once you complete it, please send the repository link. If you have any questions regarding the assignment, don't hesitate to contact us.

Good luck, and have fun!